

Series Temporales

Analítica de Datos



¿Qué es una Serie Temporal?

Definición:

- Secuencia de observaciones de una variable medidas a lo largo del tiempo
- Intervalos regulares (día, semana, mes, año)
- El orden de las observaciones importa
- Existe dependencia temporal entre ellas

Diferencia clave:

- En datos tradicionales: cada observación es independiente
- En series temporales: el valor en t depende de valores anteriores

Ejemplos de Series Temporales

- **Finanzas:** Precio de acciones, tipo de cambio, índices bursátiles
- **Economía:** PBI trimestral, tasa de desempleo, inflación mensual
- **Retail:** Ventas diarias de productos, demanda de servicios
- **Clima:** Temperatura diaria, precipitaciones mensuales
- **Tecnología:** Tráfico web, uso de CPU, número de usuarios activos

Características Principales

1. **Dependencia temporal:** El valor en el tiempo t puede depender de valores en tiempos anteriores $t - 1, t - 2, \dots$
2. **Autocorrelación:** La correlación de una variable consigo misma en diferentes momentos del tiempo
3. **Orden:** El orden de las observaciones no puede alterarse sin perder información

Análisis Descriptivo vs Predictivo

Análisis Descriptivo:

- Entender patrones históricos
- Identificar estacionalidades
- Detectar anomalías
- Descomponer la serie

Análisis Predictivo:

- Predecir valores futuros
- Estimar demanda futura
- Proyectar tendencias
- Generar intervalos de confianza

Ambos enfoques son complementarios

Componentes de una Serie Temporal

Una serie temporal Y_t se puede descomponer en cuatro componentes:

- **Tendencia (T_t):** Comportamiento de largo plazo (crecimiento o decrecimiento sostenido)
- **Ciclo (C_t):** Fluctuaciones de mediano plazo sin frecuencia fija
- **Estacionalidad (S_t):** Patrones que se repiten con periodicidad fija
- **Ruido (ϵ_t):** Componente aleatoria impredecible

Modelos de Descomposición

Modelo Aditivo:

$$Y_t = T_t + S_t + \epsilon_t$$

Variaciones estacionales constantes a lo largo del tiempo

Ejemplo: Si las ventas aumentan siempre en 1000 unidades en diciembre, independientemente de si el nivel base es 5000 o 50000 unidades

Modelo Multiplicativo:

$$Y_t = T_t \times S_t \times \epsilon_t$$

Variaciones estacionales proporcionales al nivel de la serie

Ejemplo: Si las ventas aumentan un 20 % en diciembre, la cantidad absoluta depende del nivel base: 1000 unidades extra si el nivel es 5000, pero 10000 unidades extra si el nivel es 50000

Fórmula:

$$\text{SMA}_k(t) = \frac{1}{k} \sum_{i=0}^{k-1} Y_{t-i} = \frac{Y_t + Y_{t-1} + \cdots + Y_{t-k+1}}{k}$$

Predicción:

$$\hat{Y}_{t+1} = \text{SMA}_k(t)$$

Limitación de la Media Móvil Simple

Importante: Para predecir más de un paso adelante, todas las predicciones usan el mismo valor:

$$\hat{Y}_{t+h} = \text{SMA}_k(t) \quad \text{para cualquier } h \geq 1$$

- Solo útil para predicciones a muy corto plazo (1-2 pasos)
- No captura tendencia ni estacionalidad en predicciones futuras

k pequeño (3-5):

- Responde rápido a cambios
- Más sensible al ruido

k grande (10-20):

- Suaviza más
- Responde lento a cambios reales

Ejercicio: Cálculo Manual de Media Móvil

Ventas diarias (en unidades):

Día	1	2	3	4	5	6	7	8	9	10
Ventas	23	27	24	30	28	26	31	29	27	33

Calcular media móvil de orden $k = 3$

Ejercicio: Cálculos

- **Día 3:** $SMA_3(3) = \frac{23+27+24}{3} = 24,67$
- **Día 4:** $SMA_3(4) = \frac{27+24+30}{3} = 27,00$
- **Día 5:** $SMA_3(5) = \frac{24+30+28}{3} = 27,33$
- **Día 6:** $SMA_3(6) = \frac{30+28+26}{3} = 28,00$
- **Día 7:** $SMA_3(7) = \frac{28+26+31}{3} = 28,33$
- **Día 8:** $SMA_3(8) = \frac{26+31+29}{3} = 28,67$
- **Día 9:** $SMA_3(9) = \frac{31+29+27}{3} = 29,00$
- **Día 10:** $SMA_3(10) = \frac{29+27+33}{3} = 29,67$

Predicción día 11: $\hat{Y}_{11} = 29,67 \approx 30$ unidades

Fórmula:

$$\text{WMA}_k(t) = \sum_{i=0}^{k-1} w_i \cdot Y_{t-i}$$

donde $\sum w_i = 1$ y típicamente

$$w_0 > w_1 > \dots > w_{k-1}$$

Ejemplo para $k = 3$: $w_0 = 0,5$, $w_1 = 0,3$, $w_2 = 0,2$

Implementación: Imports y Datos

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Datos del ejemplo
6 ventas = [23, 27, 24, 30, 28, 26, 31, 29, 27, 33]
7 dias = list(range(1, 11))
8
9 # Crear serie temporal
10 ts = pd.Series(ventas, index=dias)
```

Implementación: Cálculo y Predicción

```
1 # Media movil simple de orden 3
2 sma_3 = ts.rolling(window=3).mean()
3
4 # Media movil simple de orden 5
5 sma_5 = ts.rolling(window=5).mean()
6
7 # Prediccion para el proximo periodo
8 prediccion = sma_3.iloc[-1]
9 print(f"Predicción para día 11: {prediccion:.2f}")
```

Implementación: Visualización

```
1 # Visualizacion
2 plt.figure(figsize=(10, 5))
3 plt.plot(dias, ventas, 'o-', label='Ventas Reales', linewidth=2)
4 plt.plot(dias, sma_3, 's--', label='SMA(3)', linewidth=2)
5 plt.plot(dias, sma_5, '^--', label='SMA(5)', linewidth=2)
6 plt.xlabel('Dia')
7 plt.ylabel('Ventas')
8 plt.title('Ventas con Medias Moviles')
9 plt.legend()
10 plt.grid(True, alpha=0.3)
11 plt.show()
```


Ejercicio: Descomposición Manual

Ventas trimestrales (en miles de unidades) durante 3 años:

Año	Q1	Q2	Q3	Q4
2021	10	12	11	15
2022	11	13	12	16
2023	12	14	13	17

Descomponer usando modelo aditivo

Paso 1: Calcular la Tendencia

Trimestre	Cálculo	Tendencia T_t
2021-Q1	-	-
2021-Q2	$\frac{10+12+11+15}{4}$	12.0
2021-Q3	$\frac{12+11+15+11}{4}$	12.25
2021-Q4	$\frac{11+15+11+13}{4}$	12.25
2022-Q1	$\frac{15+11+13+12}{4}$	12.5
2022-Q2	$\frac{11+13+12+16}{4}$	13.0
2022-Q3	$\frac{13+12+16+12}{4}$	13.25
2022-Q4	$\frac{12+16+12+14}{4}$	13.5
2023-Q1	$\frac{16+12+14+13}{4}$	13.75
2023-Q2	$\frac{12+14+13+17}{4}$	14.0

Media móvil centrada de 4 períodos (1 año completo)

Paso 2: Componente Estacional + Ruido

Trimestre	Ventas	Tendencia T_t	$Y_t - T_t$
2021-Q2	12	12.0	0.0
2021-Q3	11	12.25	-1.25
2021-Q4	15	12.25	2.75
2022-Q1	11	12.5	-1.5
2022-Q2	13	13.0	0.0
2022-Q3	12	13.25	-1.25
2022-Q4	16	13.5	2.5
2023-Q1	12	13.75	-1.75
2023-Q2	14	14.0	0.0

Restamos la tendencia de los valores observados

Paso 3: Índices Estacionales

Trimestre	Índice Estacional S_t
Q1	$\frac{-1,5+(-1,75)}{2} = -1,625$
Q2	$\frac{0,0+0,0+0,0}{3} = 0,0$
Q3	$\frac{-1,25+(-1,25)}{2} = -1,25$
Q4	$\frac{2,75+2,5}{2} = 2,625$

Promediamos las diferencias para cada trimestre

Interpretación de la Descomposición

- **Tendencia:** Crecimiento sostenido de aproximadamente 0.25 mil unidades por trimestre
- **Q1 y Q3:** Temporadas bajas (ventas por debajo de la tendencia)
- **Q4:** Temporada alta con aproximadamente 2.6 mil unidades adicionales
- **Q2:** Se mantiene cerca de la tendencia

Fórmula:

$$\hat{Y}_{t+1} = \alpha Y_t + (1 - \alpha) \hat{Y}_t$$

donde $\alpha \in [0, 1]$ es el parámetro de suavizamiento

- α cercano a 1: más peso a observaciones recientes
- α cercano a 0: más peso a valores históricos

Método de Holt (Tendencia Lineal)

Componentes:

- **Nivel (L_t):** Valor base estimado de la serie
- **Tendencia (T_t):** Cambio (pendiente) del nivel

Ecuaciones:

$$L_t = \alpha Y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

$$\hat{Y}_{t+h} = L_t + h \cdot T_t$$

donde α controla el nivel y β controla la tendencia

Cuando la estacionalidad es constante:

$$L_t = \alpha(Y_t - S_{t-s}) + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

$$S_t = \gamma(Y_t - L_t) + (1 - \gamma)S_{t-s}$$

$$\hat{Y}_{t+h} = L_t + h \cdot T_t + S_{t+h-s}$$

donde s es la longitud del período estacional (ej: 12 para datos mensuales)

Cuando la estacionalidad crece con el nivel:

$$L_t = \alpha \frac{Y_t}{S_{t-s}} + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

$$S_t = \gamma \frac{Y_t}{L_t} + (1 - \gamma)S_{t-s}$$

$$\hat{Y}_{t+h} = (L_t + h \cdot T_t) \times S_{t+h-s}$$

Interpretación de los Parámetros

¿De dónde salen α , β y γ ?

Se calculan automáticamente al ajustar el modelo. El software (por ejemplo, statsmodels en Python) encuentra los valores que minimizan el error de predicción en los datos históricos.

Interpretación:

- α (**nivel**): Qué tan rápido se adapta al valor actual. Valores altos (0.7-0.9) reaccionan rápido a cambios
- β (**tendencia**): Qué tan rápido se adapta a cambios en la tendencia. Valores bajos (0.1-0.3) evitan fluctuaciones
- γ (**estacionalidad**): Qué tan rápido se adapta a cambios en patrones estacionales. Valores bajos (0.1-0.2) mantienen patrones estables

Holt-Winters: Imports y Datos

```
1 from statsmodels.tsa.holtwinters import ExponentialSmoothing
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Datos mensuales con estacionalidad (24 meses)
7 ventas = [112, 118, 132, 129, 121, 135, 148, 148, 136, 119,
8           104, 118, 115, 126, 141, 135, 125, 149, 170, 170,
9           158, 133, 114, 140]
10 fechas = pd.date_range('2022-01', periods=24, freq='MS')
11 ts = pd.Series(ventas, index=fechas)
```

Holt-Winters: Ajuste del Modelo

```
1 # Ajustar modelo Holt-Winters
2 # seasonal_periods=12 indica estacionalidad anual
3 # trend='add': tendencia aditiva
4 # seasonal='add': estacionalidad aditiva
5 modelo = ExponentialSmoothing(
6     ts,
7     seasonal_periods=12,
8     trend='add',
9     seasonal='add'
10 )
11
12 # Entrenar el modelo
13 fit = modelo.fit()
```

Holt-Winters: Predicciones y Parámetros

```
1 # Hacer predicciones para los proximos 6 meses
2 predicciones = fit.forecast(steps=6)
3
4 print("Parametros estimados:")
5 print(f"Alpha (nivel): {fit.params['smoothing_level']:.3f}")
6 print(f"Beta (tendencia): {fit.params['smoothing_trend']:.3f}")
7 print(f"Gamma (estacionalidad): {fit.params['smoothing_seasonal']:.3f}")
8
9 print("\nPredicciones para los proximos 6 meses:")
10 print(predicciones)
```

Holt-Winters: Visualización

```
1 # Visualizacion
2 plt.figure(figsize=(12, 6))
3 plt.plot(ts, label='Datos Historicos', marker='o')
4 plt.plot(fit.fittedvalues, label='Ajuste del Modelo',
5         linestyle='--', linewidth=2)
6 plt.plot(predicciones, label='Predicciones',
7         marker='s', linestyle='--', linewidth=2)
8 plt.xlabel('Fecha')
9 plt.ylabel('Ventas')
10 plt.title('Modelo Holt-Winters')
11 plt.legend()
12 plt.grid(True, alpha=0.3)
13 plt.show()
```

Cuándo usar Holt-Winters

- Series con tendencia y estacionalidad bien definidas
- Datos mensuales, trimestrales o semanales con patrones repetitivos
- Pronósticos a corto o mediano plazo (hasta uno o dos ciclos estacionales)
- Sectores como retail, turismo o energía con patrones estacionales marcados

Limitaciones de Holt-Winters

- Supone que los patrones del pasado se mantendrán en el futuro
- No considera el impacto de variables externas (promociones, cambios económicos)
- La incertidumbre de las predicciones aumenta rápidamente con el horizonte

AutoRegressive Integrated Moving Average

Familia de modelos estadísticos para análisis y predicción de series temporales

Más sofisticado que métodos anteriores

Puede capturar una mayor variedad de patrones

Definición: Una serie es estacionaria cuando sus propiedades estadísticas (media, varianza) no cambian con el tiempo

Señales de no estacionaridad:

- Tendencia clara (creciente o decreciente)
- Varianza que cambia con el tiempo
- Estacionalidad fuerte

Primera diferencia:

$$\Delta Y_t = Y_t - Y_{t-1}$$

Segunda diferencia:

$$\Delta^2 Y_t = \Delta Y_t - \Delta Y_{t-1} = (Y_t - Y_{t-1}) - (Y_{t-1} - Y_{t-2})$$

¿Cuántas diferencias usar?

- $d = 1$: cuando la serie tiene tendencia lineal
- $d = 2$: cuando después de la primera diferencia todavía hay tendencia

Componente Autoregresivo

$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + \epsilon_t$$

El valor actual depende de los últimos p valores pasados

Los coeficientes $\phi_1, \dots, \phi_p$ se calculan automáticamente al ajustar el modelo

Componente de Integración

Indica cuántas veces diferenciamos la serie para hacerla estacionaria:

- $d = 0$: La serie ya es estacionaria
- $d = 1$: Aplicamos primera diferencia
- $d = 2$: Aplicamos segunda diferencia (raro en la práctica)

Componente de Media Móvil

$$Y_t = \mu + \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2} + \cdots + \theta_q\epsilon_{t-q}$$

El valor actual depende de shocks o errores aleatorios de los últimos q períodos

Los coeficientes $\theta_1, \dots, \theta_q$ se calculan automáticamente

Combinando todo:

$$\Delta^d Y_t = c + \sum_{i=1}^p \phi_i \Delta^d Y_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t$$

Definido sobre la serie diferenciada d veces

Determinar d (diferenciación)

- **Visualizar la serie:** Si tiene tendencia, probablemente necesite $d = 1$ o $d = 2$
- **Pruebas estadísticas:** Tests como ADF (Augmented Dickey-Fuller) o KPSS indican si la serie es estacionaria
- **Regla práctica:** Empezar con $d = 1$ y verificar si la serie diferenciada parece estacionaria

¿Qué son?

Son métricas que miden la dependencia temporal de una serie. Nos ayudan a entender cómo se relaciona cada valor con sus valores pasados.

ACF (Autocorrelation Function):

- Muestra la correlación entre Y_t y Y_{t-k} para diferentes lags k
- Si decae lentamente, sugiere diferenciación o términos MA

PACF (Partial Autocorrelation Function):

- Muestra la correlación directa entre Y_t y Y_{t-k} controlando por valores intermedios
- Picos significativos sugieren términos AR

¿Qué son?

Son métricas que nos permiten comparar diferentes modelos ARIMA. Balancean el ajuste del modelo (qué tan bien explica los datos) con su complejidad (número de parámetros). Nos ayudan a elegir el mejor modelo entre varias opciones.

AIC (Akaike Information Criterion):

- Penaliza modelos complejos pero menos que BIC
- Útil para comparar modelos con diferentes números de parámetros
- Menor AIC indica mejor modelo

BIC (Bayesian Information Criterion):

- Penaliza más la complejidad que AIC

- Implementado en Python con `pmdarima`
- Prueba automáticamente diferentes combinaciones de p , d y q
- Selecciona el modelo con mejor AIC
- Muy útil cuando no estamos seguros de los valores
- Es importante entender qué está haciendo para interpretar los resultados

ARIMA: Imports y Datos

```
1 from statsmodels.tsa.arima.model import ARIMA
2 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 # Datos de ejemplo: ventas mensuales
8 ventas = [200, 205, 210, 215, 218, 220, 225, 230, 232, 235,
9           240, 245, 242, 248, 253, 258, 260, 265, 270, 275,
10          278, 280, 285, 290]
11 fechas = pd.date_range('2022-01', periods=24, freq='MS')
12 ts = pd.Series(ventas, index=fechas)
```

ARIMA: Visualizar ACF y PACF

```
1 # Visualizar ACF y PACF para diagnostico
2 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
3 plot_acf(ts, ax=axes[0], lags=10)
4 plot_pacf(ts, ax=axes[1], lags=10)
5 plt.show()
```

ARIMA: Ajustar Modelo

```
1 # Ajustar modelo ARIMA(1,1,1)
2 # p=1: un termino autoregresivo
3 # d=1: una diferenciacion
4 # q=1: un termino de media movil
5 modelo = ARIMA(ts, order=(1, 1, 1))
6 fit = modelo.fit()
7
8 # Resumen del modelo
9 print(fit.summary())
```

ARIMA: Predicciones

```
1 # Hacer predicciones para los proximos 6 meses
2 predicciones = fit.forecast(steps=6)
3 print("\nPredicciones:")
4 print(predicciones)
```

ARIMA: Visualización

```
1 # Visualizacion
2 plt.figure(figsize=(12, 6))
3 plt.plot(ts, label='Datos Historicos', marker='o')
4 plt.plot(fit.fittedvalues, label='Ajuste ARIMA',
5         linestyle='--', linewidth=2)
6 plt.plot(predicciones, label='Predicciones',
7         marker='s', linestyle='--', linewidth=2)
8 plt.xlabel('Fecha')
9 plt.ylabel('Ventas')
10 plt.title('Modelo ARIMA(1,1,1)')
11 plt.legend()
12 plt.grid(True, alpha=0.3)
13 plt.show()
```


Auto ARIMA

```
1 from pmdarima import auto_arima
2
3 # Buscar el mejor modelo automaticamente
4 modelo_auto = auto_arima(
5     ts,
6     start_p=0, max_p=3,
7     start_q=0, max_q=3,
8     d=None, # Se determina automaticamente
9     seasonal=False,
10    trace=True, # Muestra el proceso de busqueda
11    error_action='ignore',
12    suppress_warnings=True,
13    stepwise=True
14 )
15
16 print(modelo_auto.summary())
```

Auto ARIMA: Predicciones

```
1 # Predicciones
2 predicciones_auto = modelo_auto.predict(n_periods=6)
3 print("\nPredicciones con Auto ARIMA:")
4 print(predicciones_auto)
```

¡IMPORTANTE!

En series temporales **NO podemos mezclar los datos**

Debemos respetar el orden temporal

Train-Test Split: Correcto vs Incorrecto

```
1 # CORRECTO para series temporales
2 train_size = int(len(ts) * 0.8)
3 train = ts[:train_size]
4 test = ts[train_size:]
5
6 # INCORRECTO (no hacer esto)
7 # from sklearn.model_selection import train_test_split
8 # train, test = train_test_split(ts, test_size=0.2) # NO!
```

- **MAE (Mean Absolute Error):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

- **MSE (Mean Squared Error):**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

- **RMSE (Root Mean Squared Error):**

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$$

- **MAPE (Mean Absolute Percentage Error):**

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{Y_i - \hat{Y}_i}{Y_i} \right|$$

¡IMPORTANTE!

NO podemos usar cross-validation tradicional

Mezclar los datos violaría la dependencia temporal.
Estamos con información del futuro alimentando al
pasado.

Usamos **Time Series Cross-Validation**

Rolling Window (Ventana Móvil)

El tamaño del train se mantiene constante

En cada fold, “deslizamos” la ventana hacia adelante

- **Fold 1:** Entrenar con meses 1-6, probar con meses 7-8
- **Fold 2:** Entrenar con meses 3-8, probar con meses 9-10
- **Fold 3:** Entrenar con meses 5-10, probar con meses 11-12

El tamaño del entrenamiento siempre es 6 meses

Expanding Window (Ventana Expansiva)

El tamaño del train crece en cada fold

El inicio se mantiene fijo y solo avanzamos el final

- **Fold 1:** Entrenar con meses 1-4, probar con meses 5-6
- **Fold 2:** Entrenar con meses 1-6, probar con meses 7-8
- **Fold 3:** Entrenar con meses 1-8, probar con meses 9-10
- **Fold 4:** Entrenar con meses 1-10, probar con meses 11-12

Siempre empezamos desde el mes 1

¿Cuándo usar cada una?

- **Rolling Window:** Útil cuando queremos evaluar el desempeño con una cantidad fija de datos históricos. Simula mejor el escenario real donde tenemos una ventana de tiempo limitada
- **Expanding Window:** Útil cuando queremos aprovechar toda la información histórica disponible. Simula mejor el caso donde acumulamos datos a lo largo del tiempo

TimeSeriesSplit: Setup

```
1 from sklearn.model_selection import TimeSeriesSplit
2 from sklearn.metrics import mean_absolute_error, mean_squared_error
3 import numpy as np
4
5 # Configurar cross-validation temporal
6 # n_splits=5 crea 5 folds
7 tscv = TimeSeriesSplit(n_splits=5)
8
9 mae_scores = []
10 rmse_scores = []
```

TimeSeriesSplit: Loop

```
1 for fold, (train_index, test_index) in enumerate(tscv.split(ts)):
2     train = ts.iloc[train_index]
3     test = ts.iloc[test_index]
4
5     print(f"Fold {fold+1}: Train size={len(train)}, Test size={len(test)}")
6
7     # Entrenar modelo (ejemplo con ARIMA)
8     modelo = ARIMA(train, order=(1,1,1))
9     fit = modelo.fit()
10
11     # Predecir
12     predicciones = fit.forecast(steps=len(test))
```

TimeSeriesSplit: Métricas

```
1  # Calcular metricas
2  mae = mean_absolute_error(test, predicciones)
3  rmse = np.sqrt(mean_squared_error(test, predicciones))
4
5  mae_scores.append(mae)
6  rmse_scores.append(rmse)
7  print(f"  MAE: {mae:.2f}, RMSE: {rmse:.2f}")
8
9  print(f"\nMAE promedio: {np.mean(mae_scores):.2f} (+/-
    {np.std(mae_scores):.2f})")
10 print(f"RMSE promedio: {np.mean(rmse_scores):.2f} (+/-
    {np.std(rmse_scores):.2f})")
```

Expanding Window: Implementación Manual

```
1 # Expanding window: el train crece en cada iteracion
2 train_size_initial = int(len(ts) * 0.3) # Empezar con 30% de los datos
3 n_folds = 5
4
5 mae_scores = []
6 rmse_scores = []
7
8 for i in range(n_folds):
9     # El train crece en cada iteracion
10    train_end = train_size_initial + i * (len(ts) - train_size_initial) //
        n_folds
11    test_start = train_end
12    test_end = train_end + (len(ts) - train_size_initial) // n_folds
13
14    train = ts[:train_end]
15    test = ts[test_start:test_end]
```

Expanding Window: Evaluación

```
1  # Entrenar y evaluar modelo
2  modelo = ARIMA(train, order=(1,1,1))
3  fit = modelo.fit()
4  predicciones = fit.forecast(steps=len(test))
5
6  mae = mean_absolute_error(test, predicciones)
7  rmse = np.sqrt(mean_squared_error(test, predicciones))
8
9  mae_scores.append(mae)
10 rmse_scores.append(rmse)
```

Comparación de Modelos: Setup

```
1 import pandas as pd
2 from sklearn.metrics import mean_absolute_error, mean_squared_error
3 import numpy as np
4
5 # Dividir datos
6 train = ts[:18]
7 test = ts[18:]
```

Comparación de Modelos: Modelo 1 - Media Móvil

```
1 # Modelo 1: Media Movil Simple
2 sma_pred = train.rolling(window=3).mean().iloc[-1]
3 sma_pred = [sma_pred] * len(test) # Prediccion constante
```


Comparación de Modelos: Modelo 2 - Holt-Winters

```
1 # Modelo 2: Holt-Winters
2 from statsmodels.tsa.holtwinters import ExponentialSmoothing
3 hw_model = ExponentialSmoothing(train, trend='add', seasonal='add',
4                                 seasonal_periods=12).fit()
5 hw_pred = hw_model.forecast(steps=len(test))
```

Comparación de Modelos: Modelo 3 - ARIMA

```
1 # Modelo 3: ARIMA
2 arima_model = ARIMA(train, order=(1,1,1)).fit()
3 arima_pred = arima_model.forecast(steps=len(test))
```

Comparación de Modelos: Calcular Métricas

```
1 # Calcular metricas
2 resultados = {
3     'Modelo': ['Media Movil', 'Holt-Winters', 'ARIMA'],
4     'MAE': [
5         mean_absolute_error(test, sma_pred),
6         mean_absolute_error(test, hw_pred),
7         mean_absolute_error(test, arima_pred)
8     ],
9     'RMSE': [
10        np.sqrt(mean_squared_error(test, sma_pred)),
11        np.sqrt(mean_squared_error(test, hw_pred)),
12        np.sqrt(mean_squared_error(test, arima_pred))
13    ]
14 }
15
16 df_resultados = pd.DataFrame(resultados)
17 print(df_resultados)
```

- Los modelos estadísticos como ARIMA y Holt-Winters proporcionan intervalos de confianza
- Útiles para cuantificar la incertidumbre de las predicciones
- Típicamente se usan intervalos del 95 %

Intervalos de Confianza: ARIMA

```
1 # ARIMA con intervalos de confianza
2 predicciones_arima = fit.get_forecast(steps=6)
3 pred_mean = predicciones_arima.predicted_mean
4 pred_ci = predicciones_arima.conf_int(alpha=0.05) # 95% confianza
```

Intervalos de Confianza: Visualización

```
1 plt.figure(figsize=(12, 6))
2 plt.plot(ts, label='Datos Historicos')
3 plt.plot(pred_mean, label='Prediccion', color='red')
4 plt.fill_between(pred_ci.index,
5                  pred_ci.iloc[:, 0],
6                  pred_ci.iloc[:, 1],
7                  color='red', alpha=0.3,
8                  label='Intervalo 95%')
9 plt.legend()
10 plt.show()
```

¿Dudas?
¿Consultas?



Universidad de
SanAndrés